



HomeWellness Companion

主動感知、溫暖開口的 AI 健康伴侶 — 為獨居長者設計

IoT 感測器

Multi-Agent

Proactive AI

LangChain 1.x

Streamlit

AWS Architecture

63 Tests (TDD)

CareAgent × AnalysisAgent × AlertAgent | gpt-4o-mini · Gemini 2.5 Flash · Python 3.10+

獨居長者的健康守護缺口

54萬

台灣 65 歲以上獨居長者

2-4h

從健康異常到被發現的平均時間

1.3次

平均每位長者一天與外界的有效互動

核心問題：現有方案全是「被動等待求助」



緊急按鈕 —
昏迷時無法自行按下



家屬 LINE —
長者不主動說「我不舒服」



定時電話 —
危機在兩通之間悄悄發生



健康 App —
長者操作困難，主動性低

認識使用者：陳阿嬤



陳美蘭

78 歲 · 台北獨居 · 子女在外縣市工作

健康狀況:

高血壓，第二型糖尿病

警訊信號:

心率 >120 bpm · 血氧 <90% ·
收縮壓 >140 mmHg

行為特徵

不主動求助、怕麻煩家人、對科技產品感到焦慮。

Jobs-to-be-Done

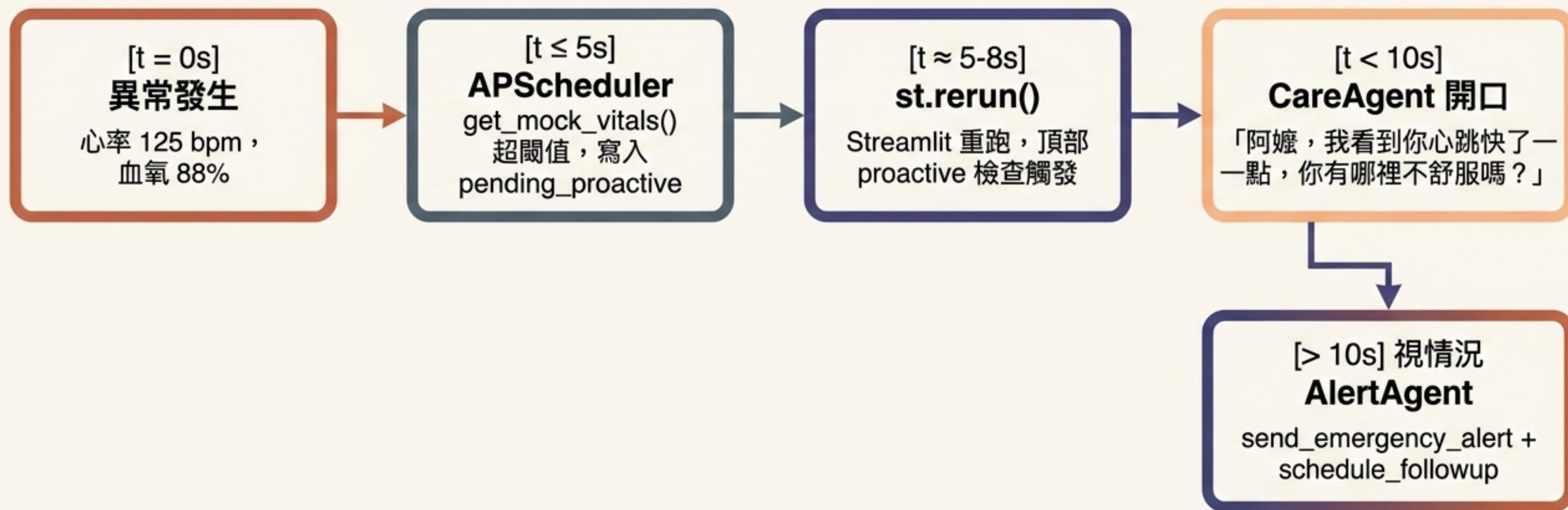
「不舒服的時候，有人主動發現並關心我——而不是我自己想辦法求助。」

Design Implication

她需要的不是「系統」，而是像孫子一樣關心她的陪伴者。

10 秒承諾 — 典範轉移：從異常到 AI 開口

Reactive (被動) → Proactive (主動)

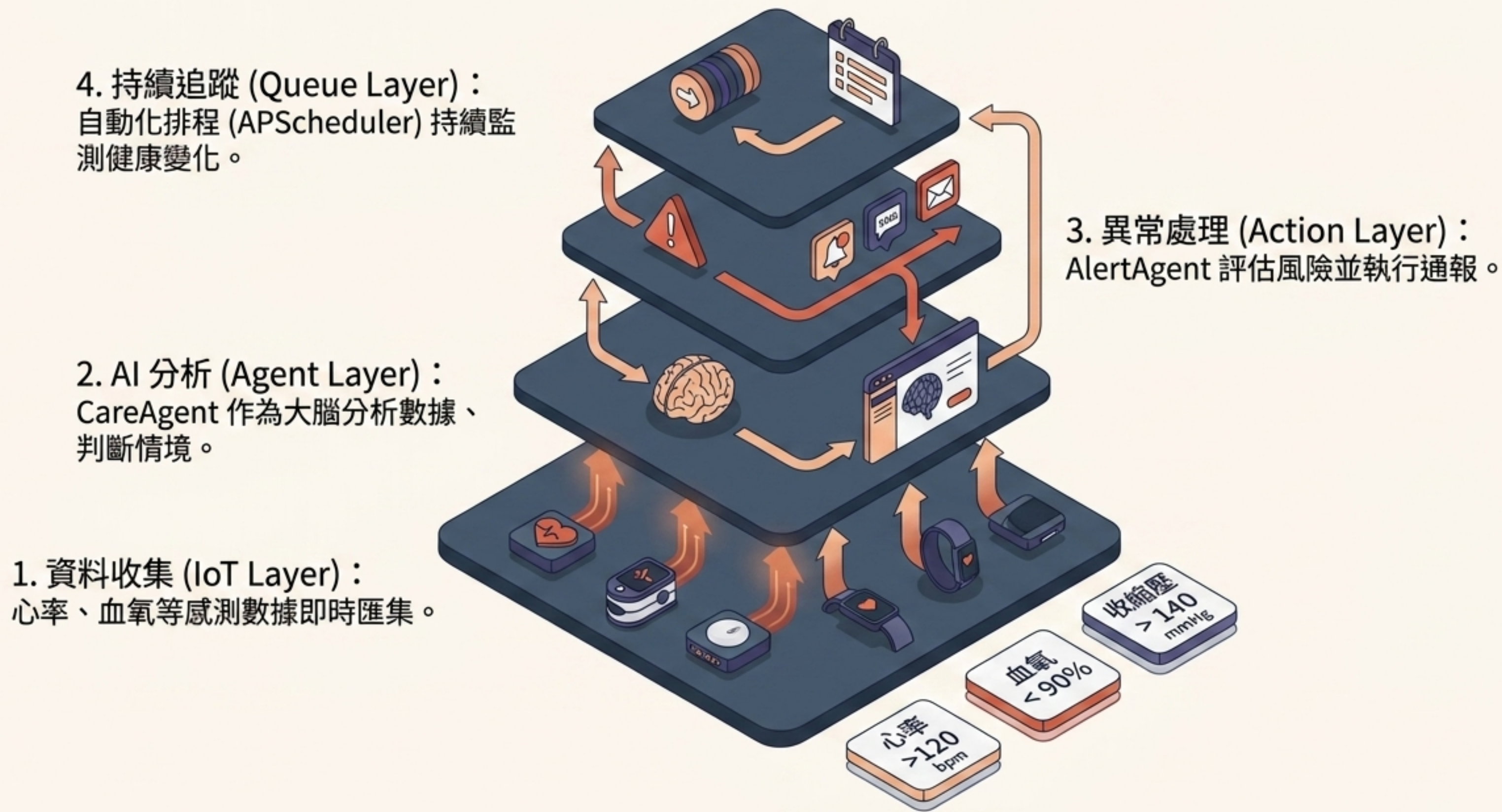


語氣設計 (Vibe Design) — 貼心長輩，不是警報系統

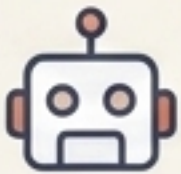
	✖ 系統警報	✔ 貼心長輩
心率異常	✖ 系統警報：「警告：心率異常，請立即就醫！」	✔ 貼心長輩：「阿嬤，心跳快了一點，你現在有哪裡不舒服嗎？」
聯絡家屬	✖ 系統警報：「已通報聯絡人。流程完成。」	✔ 貼心長輩：「我已經告訴大文了，他知道你的情況，你先放輕鬆，我陪著你。」
讀取數據	✖ 系統警報：「正在讀取血壓數據。」	✔ 貼心長輩：「頭有點暈嗎？我幫你量一下血壓，你先坐著別動。」

Prompt Engineering Rules (寫入 System Prompt)：
每則訊息只說一件事・句子不超過 2 行・用名字「阿嬤」不用「用戶」・不作醫療診斷・遇異常先安撫。

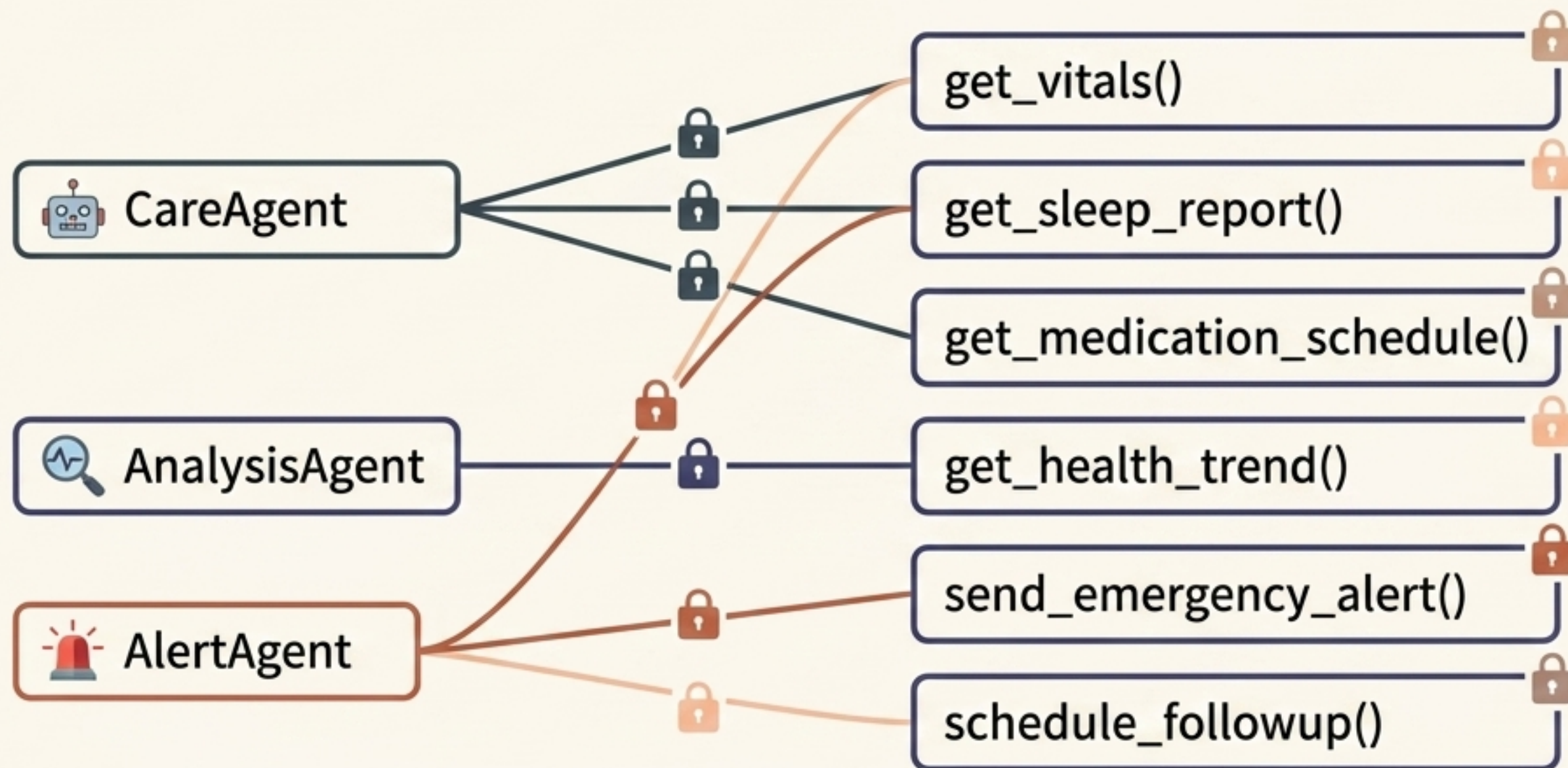
系統架構：HomeWellness 智慧守護流



Multi-Agent 協作架構 — 專業分工

 CareAgent (主控 Orchestrator)	 AnalysisAgent (深度趨勢分析)	 AlertAgent (緊急評估與通報)
模型：gpt-4o-mini (低延遲) 狀態：Session 記憶 (_AgentWithMemory) 職責：主對話、委派決策、整合子代理回應。記住對話細節，不重複詢問。	模型：Gemini 2.5 Flash (長上下文、低成本) 狀態：Stateless (無記憶) 職責：讀取近 30 天生理數據，識別異常模式並生成結構化摘要傳給 CareAgent。	模型：gpt-4o-mini (高穩定度) 狀態：Stateless (無記憶) 職責：確認最新數值，執行通報報，並使用 cron-as-tool 自排追程追蹤。

工具設計 — 最小權限原則 (Least Privilege)



Prompt Engineering Insight

Docstring = Tool Description. LangChain 把 @tool 的 docstring 傳給 LLM，讓 LLM 自主決定「何時呼叫」、「參數說明」。

核心創新：Cron-as-Tool (Agent 自排程追蹤)

1. AlertAgent 完成通報後，自主呼叫
`schedule_followup(reason, delay)`。

2. 將任務推入
`followup_queue`。

3. 背景執行緒
(APScheduler)
每 5 秒 tick 一次。

4. 取出訊息並寫入
`pending_proactive`。

5. CareAgent
主動發起後續關心對話。

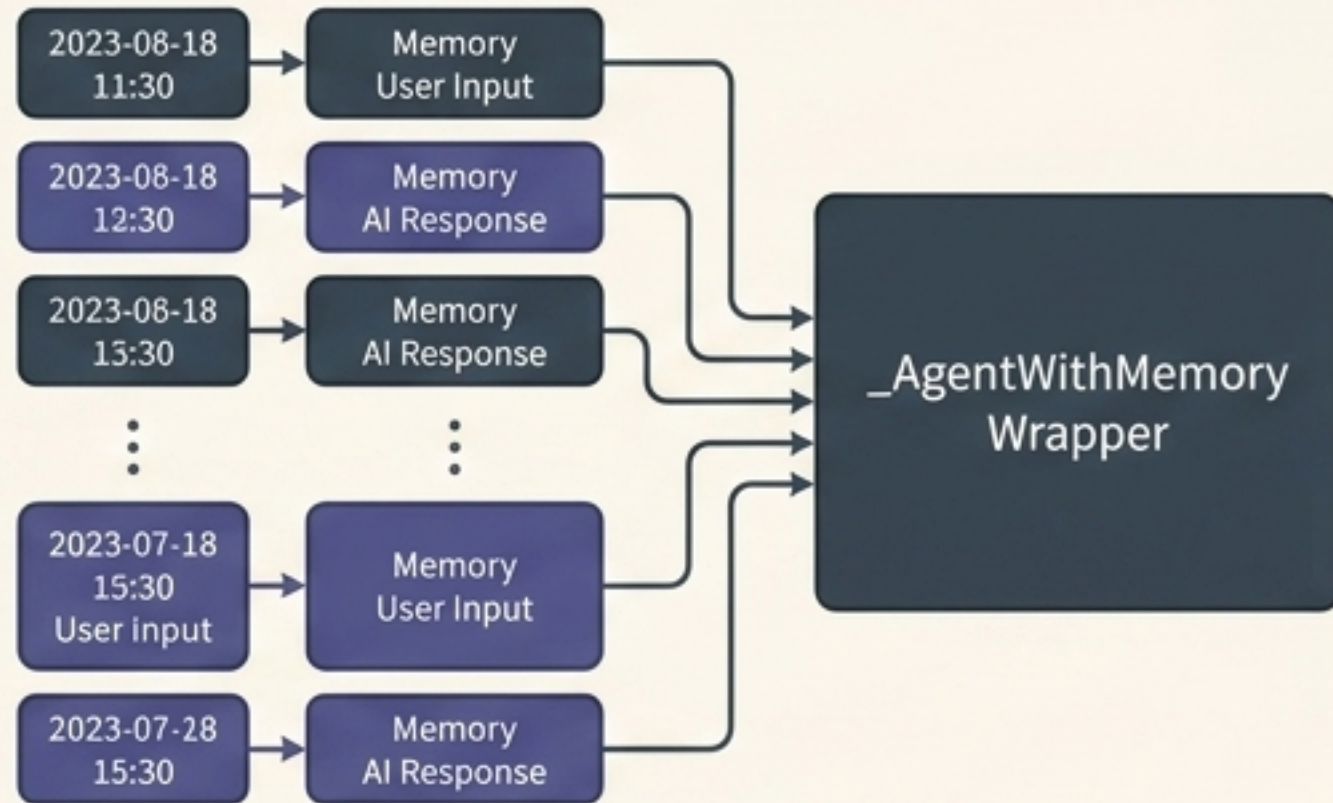
Design Tradeoff

為什麼不用寫死的 30 分鐘 Timer？

決策: 採用 Agent 自排程 (Agentic)。由 AI 判斷「緊急情況 5 分鐘後確認，輕微狀況 1 小時後確認」，而非死板基礎設施。

架構防護與記憶體管理

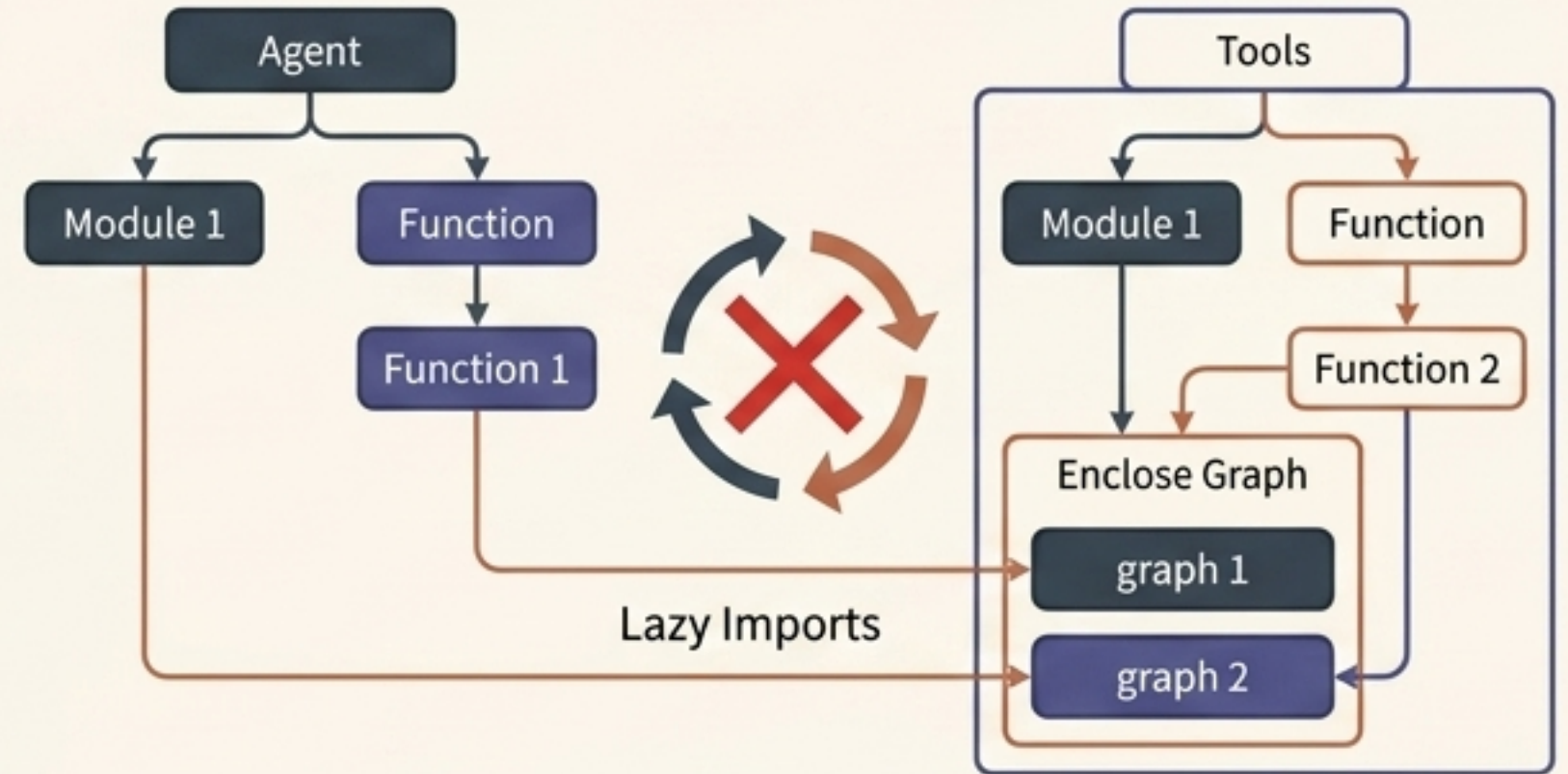
Session 記憶設計 (Memory)



技術：放棄已棄用的 `RunnableWithMessageHistory`，自建 `_AgentWithMemory wrapper`。

取捨：採用「全量歷史」而非「Sliding Window」或「Summarization」，確保不會遺漏長者在對話早期提到的隱晦症狀。

循環 Import 防護機制 (Defensive Architecture)

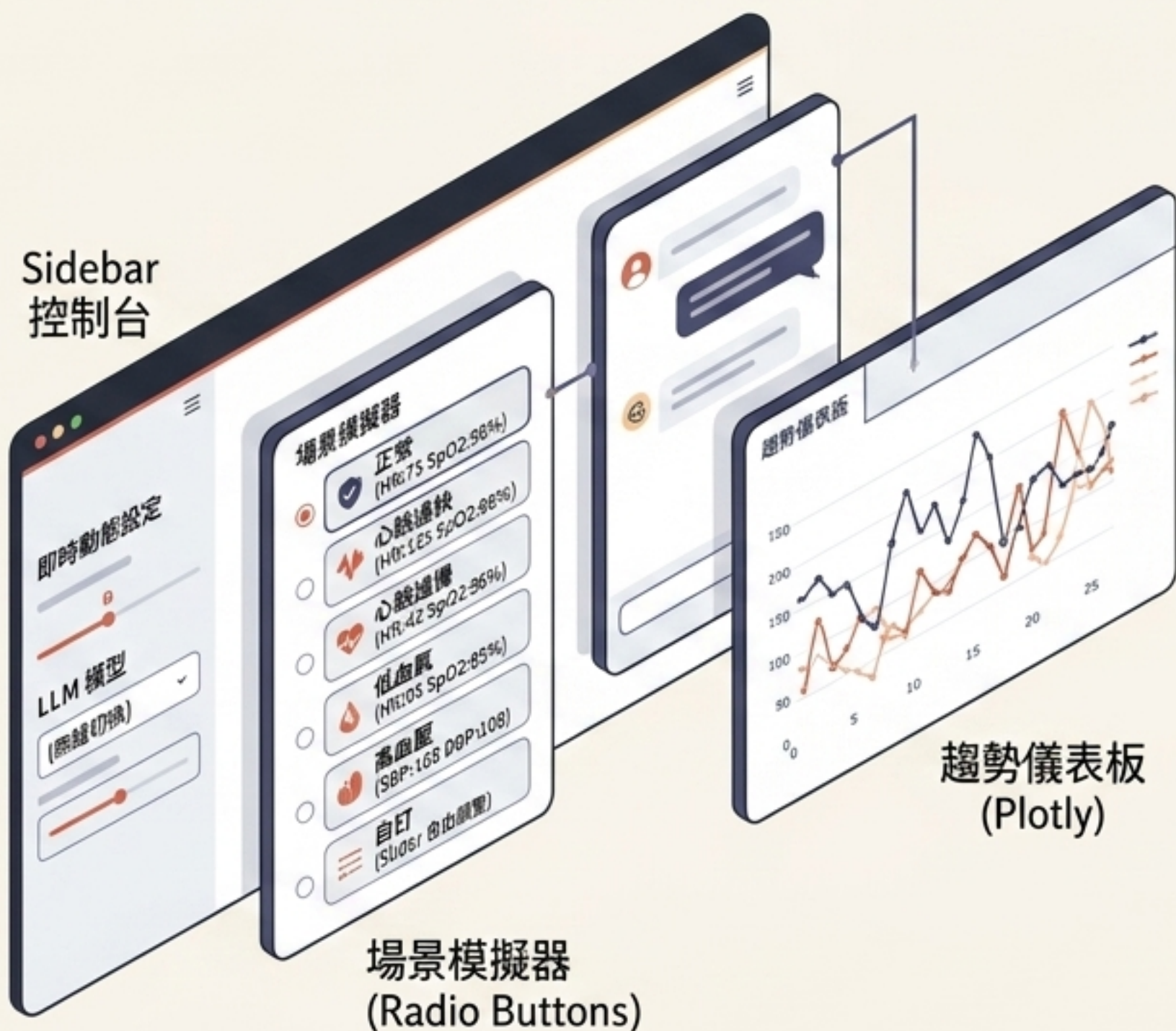


問題：Agent 與 Tools 之間容易產生依賴死結。

解法：

1. **Lazy Import**：僅在函數被呼叫時才載入模組。
2. **Closure (閉包捕捉)**：透過閉包傳遞 `graph` 實例，確保子代理為 `Stateless` 設計，每次調用皆為全新狀態。

IoT 邊緣模擬與前端架構 (Streamlit Edge)



UI Architecture

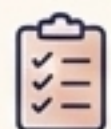
- **Sidebar 控制台**：即時動態設定 Token 監控、切換底層 LLM 模型 (無縫切換，不需重啟 Agent)。
- **場景模擬器 (Radio Buttons)**：6 種預設劇本 (正常，心跳過快，缺氧，高血壓等)，點擊瞬間覆寫 `health_profile.json`。
- **趨勢儀表板 (Plotly)**：近 30 天生理折線圖，與主對話視窗採獨立 Tab，互不干擾。

State Management

使用 `session_state` 作為訊息佇列橋接同步事件，解決背景執行緒無法直接更新 UI 的痛點。

生產準備：TDD 測試與效能指標

測試策略 (63 Tests · TDD)



測試涵蓋: Agent invoke、Tools 邊界條件、IoT mock、Prompt 關鍵字驗證。



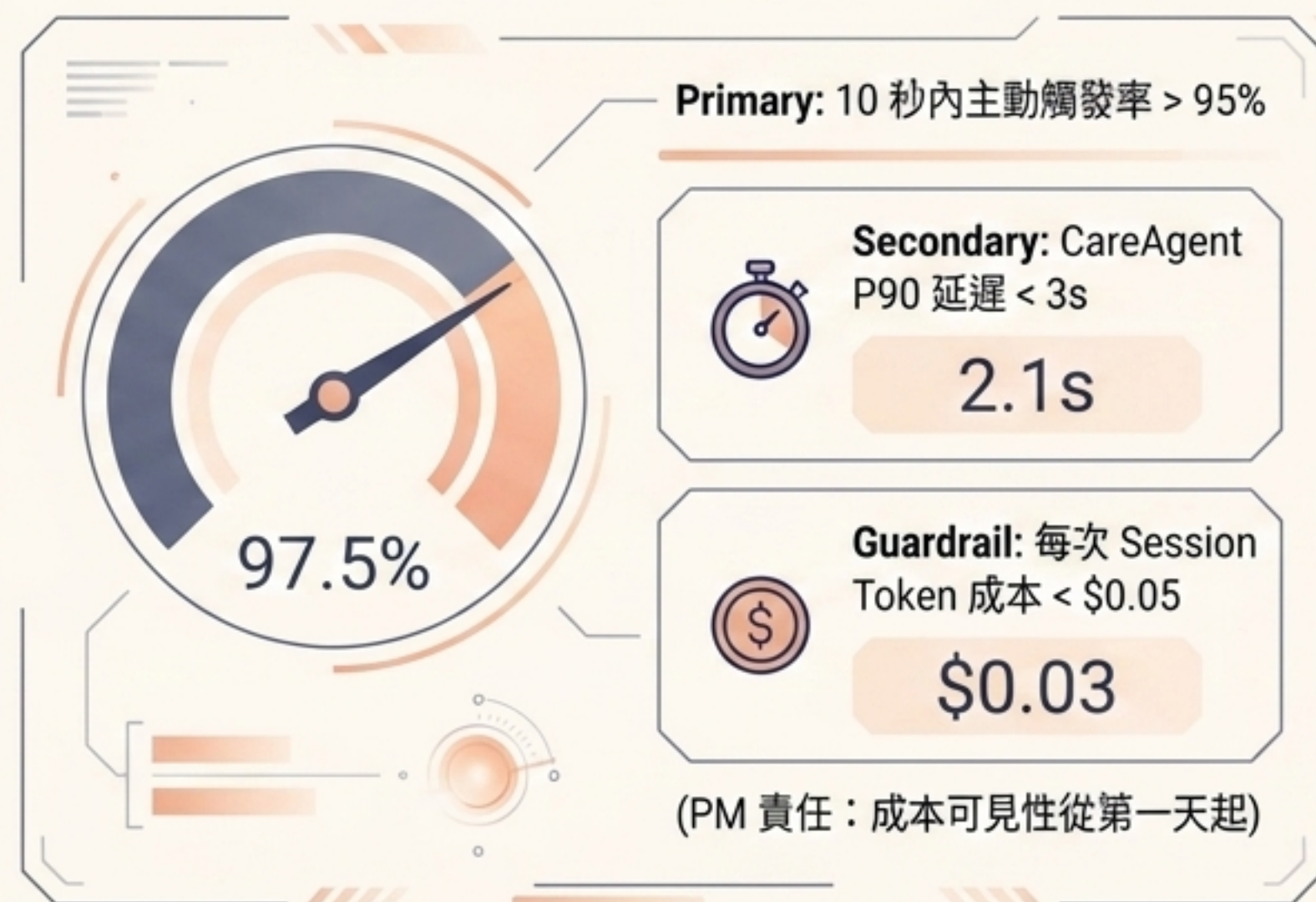
Mock 策略: 於來源模組進行 patch，配合 lazy import 確保隔離。



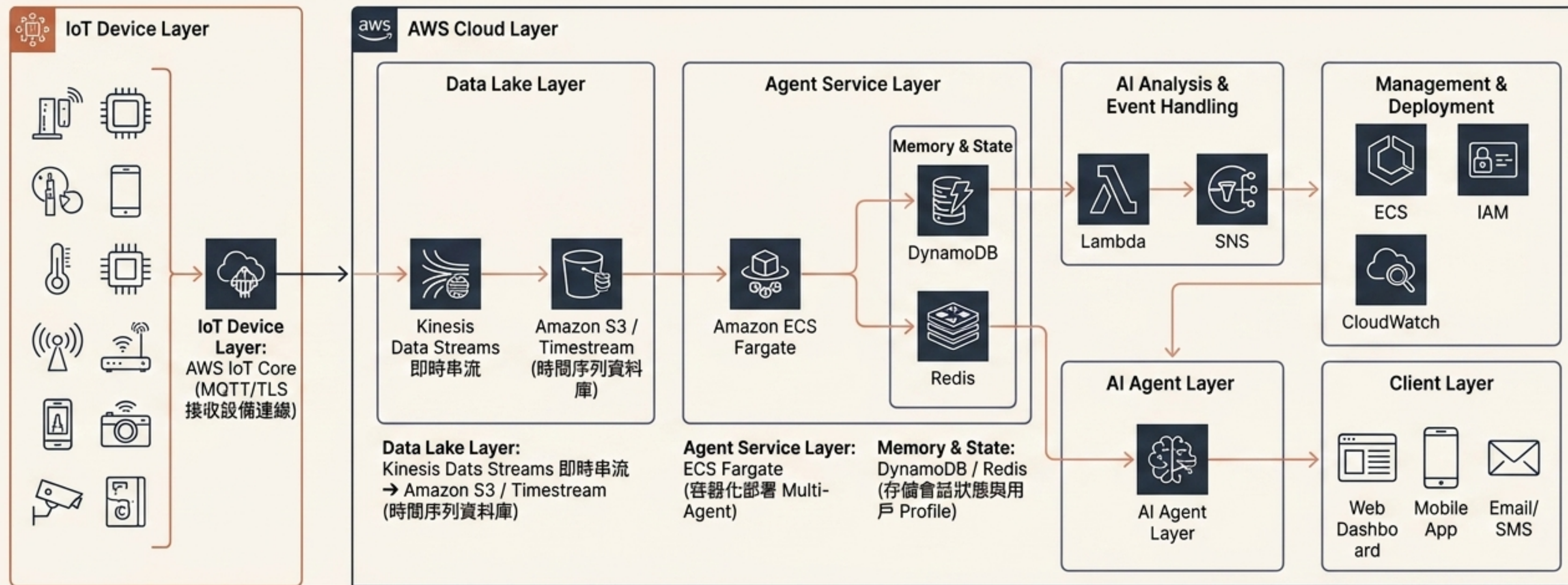
Fixture:

`@pytest.fixture(autouse=True)` 自動清空 Session，防止記憶污染。

關鍵效能指標 (Metrics HUD)



從 PoC 到企業級規模 (AWS Production Architecture)



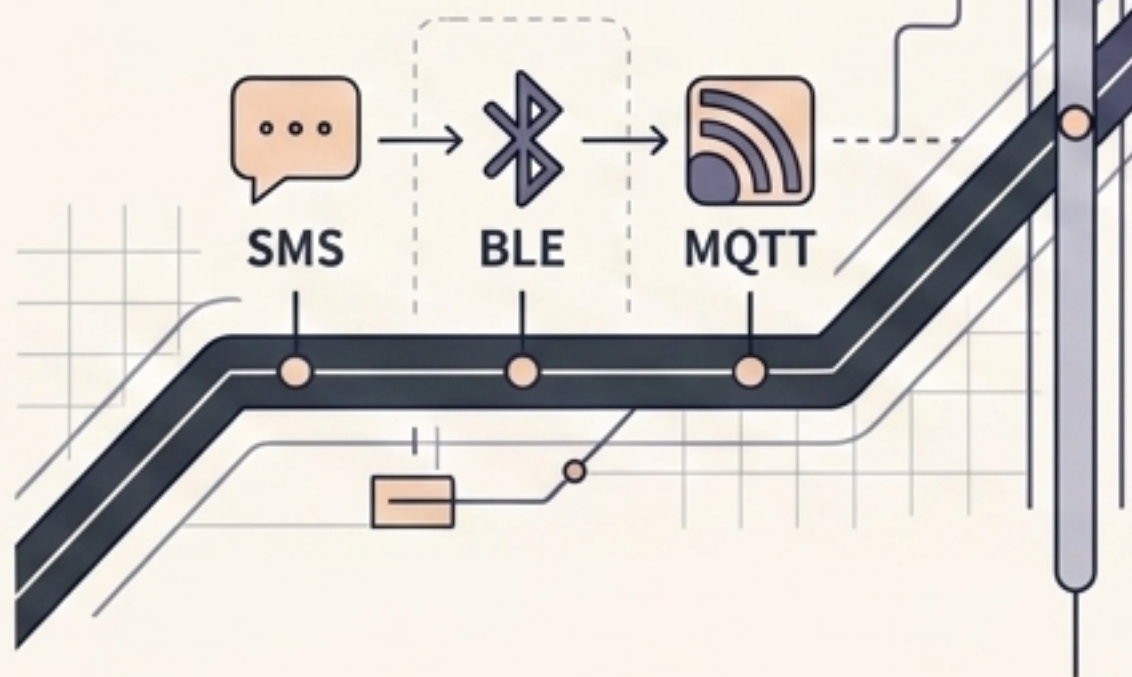
Enterprise Tradeoffs

EventBridge Cron 將取代本地 threading.Timer，確保伺服器重啟時排程不遺失。

產品推進藍圖 (Alpha → Production)

Milestone 1: Alpha (Week 1-4) 內測

- 交付: 真實 SMS 通報上線, 實際 IoT (BLE → MQTT) 串接。
- 指標: 異常偵測到通報 < 30s。



Milestone 2: Beta (Week 5-10) 小量上市

- 交付: 語音介面 (Whisper STT + TTS), 醫療資料加密 (DynamoDB), 多用戶架構。
- 指標: 長者操作成功率 > 85%, 家屬 NPS > 40。



Milestone 3: Production (Week 11-18) 正式上線

- 交付: AWS 容器化部署 (ECS + ALB), Auto Scaling。
- 指標: 系統 SLA 99.9%, 月 Token 成本 < \$0.5 / 用戶。



核心理念 Synthesis — 6 項產品設計原則

1

Proactive over Reactive

把「等待求救」的責任從長者身上移走，AI 必須主動感知。

2

Vibe First (語氣先行)

提供溫暖的陪伴與安撫，不作冷冰冰的警報系統與醫療診斷。

3

Least Privilege (最小權限)

工具精準分配，降低 AI 幻覺導致的誤觸與誤判。

4

Cron-as-Tool (自主排程)

賦予 Agent 掌控時間軸的能力，實現真正的自主行動 (Agentic)。

5

Cost Visibility

Token 效能與成本監控從 Day 1 開始，這是 PM 的基本責任。

6

Maintainability

拒絕 Deprecated API，採用 Stateless 子代理與防護機制，降低技術債。